

Modern Transfer Learning

The goal of a Model Hub is to serve as a home

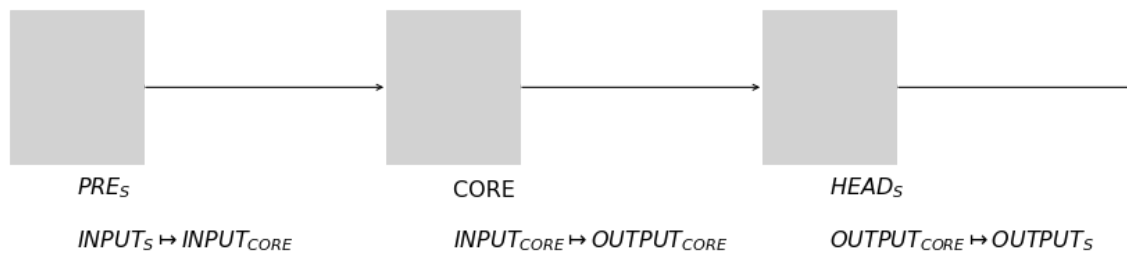
- for general purpose models
- that can be easily adapted to the user's Target task

A design that facilitates this is show below

- a Core model
- a Pre-processor for task S to use the Core model
- a Head to translate the output of the Core model to the output of Task S

In [6]: fig_tl

Out[6]:



The Core model is "task-agnostic"

- general type for its INPUT and OUTPUT

A task-specific model S

- contains the CORE mode
- adds a pre-processor PRE_S
 - translates its input type INPUT_S into the Core model's generic input type $\text{INPUT}_{\text{CORE}}$
- adds a task-specific HEAD_S
 - translate Core model's output type into task S output type

HuggingFace Model Hub

Let's see this in action on HuggingFace.

A pre-trained CORE model for the DistilBert architecture

- `class TFDistilBertPreTrainedModel(TFPreTrainedModel):`
- task agnostic

A task-specific sub-class for the Sequence Classification task

- `class`
 `TFDistilBertForSequenceClassification(TFDistilBertPreTrained`
 `TFSequenceClassificationLoss):`
- includes a HEAD specialized to Sequence Classification

The head may be customizable

- vary the number of classes

but you are free to checkout a model without a head and append your own.

In HuggingFace, a model is named by a "checkpoint"

```
checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
```

The checkpoint name is long because

- it specifies the characteristics of its training data
 - uncased, English
 - for the SST task

This Core model's checkpoint may be more difficult to adapt to

- non-English language text

Here we checkout a task-specific model

```
model =  
TFAutoModelForSequenceClassification.from_pretrained(checkpoint)
```

The TF prefix specifies that the model is a TensorFlow (rather than PyTorch) model.

The CORE model often has input type "tokens"

$$\text{INPUT}_{\text{CORE}} = \text{tokens}_{\text{CORE}}$$

- We subscript the tokens input of CORE
- because it was trained with a *specific* tokenizer

The CORE model often supplies a pre-processor from common input types to $\text{INPUT}_{\text{CORE}}$

- for example: text
 $\mapsto \text{tokens}_{\text{CORE}}$

Here we obtain a Pre-processor to convert text to the token-type of the specific checkpoint

```
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
```


Putting it all together

```
import tensorflow as tf
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = TFAutoModelForSequenceClassification.from_pretrained(checkpoint)
sequences = ["I've been waiting for a HuggingFace course my whole life.", "So h
ave I!"]

tokens = tokenizer(sequences, padding=True, truncation=True, return_tensors="t
f")
output = model(**tokens)
```

Adapting the pre-trained model: Fine-tuning

Weights

The CORE model has been pre-trained

- optimizing $\setminus \mathbf{W}_{\text{CORE}}$ for some Source task S
- using a large number of examples
- at high cost for training

The task-specific HEAD for target task T

- has untrained weights $\setminus \mathbf{W}_{\text{HEAD}}$

Fine-tuning

In order to adapt the CORE model to target task T , we *fine-tune* it

- Using a *small number* (compared to the pre-training size) of examples of the target task T

$$\langle \backslash \mathbf{x}^{(1)}, \backslash \mathbf{y}^{(1)} \rangle, \dots, \langle \backslash \mathbf{x}^{(k)}, \backslash \mathbf{y}^{(k)} \rangle$$

The purpose of fine-tuning is to

- optimize $\backslash \mathbf{W}_{\text{HEAD}}$ for target task T
- *potentially*
 - adapt $\backslash \mathbf{W}_{\text{CORE}}$ for target task T
 - be careful that fine-tuning does not completely *lose the knowledge* of the Source task
 - obtained at great expense !

In [7]: `print("Done")`

Done

